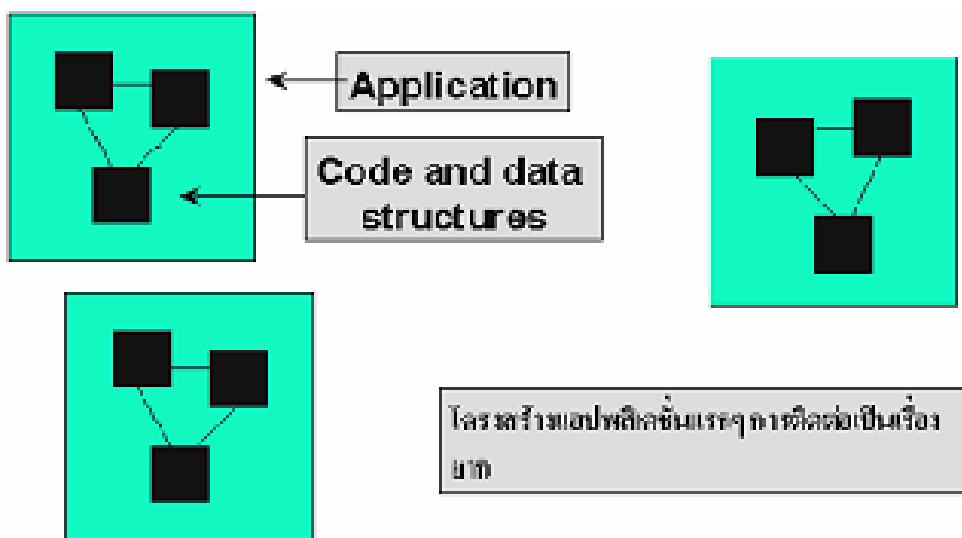


บทที่ 3

สถาปัตยกรรมของ .net

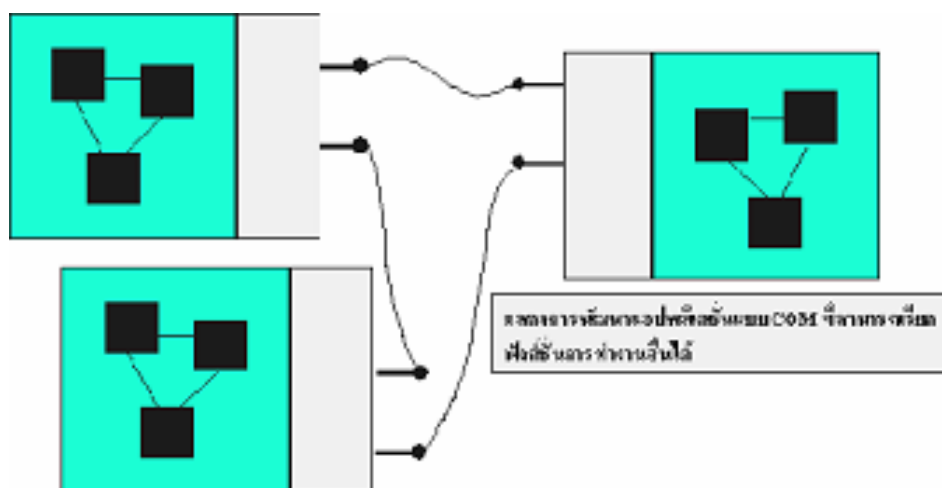
3.1 พัฒนาการของเทคโนโลยีการเขียนโปรแกรม

ก่อนที่จะมีการพัฒนาโปรแกรมเป็น Object Oriented นั้น แอปพลิเคชันแต่ละตัวเปรียบเสมือนกล่องภายในแอปพลิเคชัน ก็มีโค้ด (code) มีโครงสร้างข้อมูล (data structure) ต่างๆ ของตัวเองมีฟังก์ชันต่างๆ ของตัวแอปพลิเคชันนั้นๆ



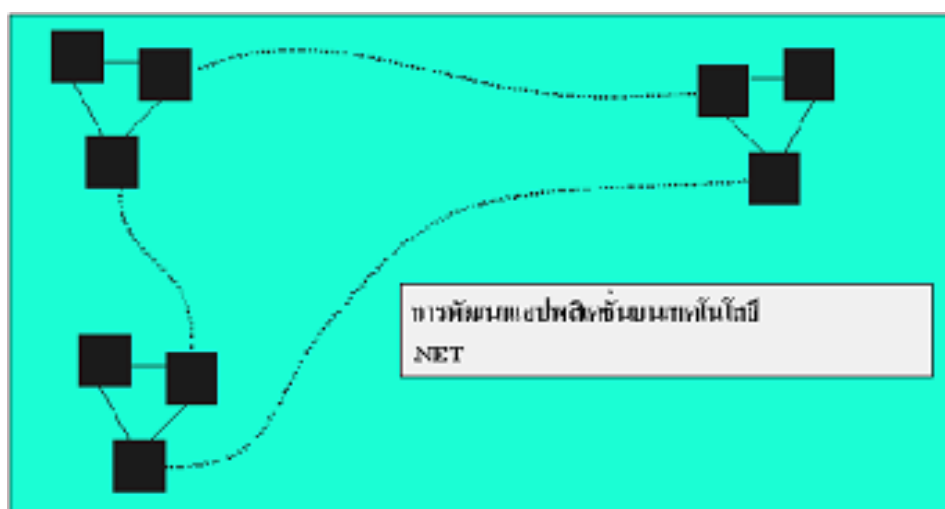
รูปที่ 3-1 พัฒนาการของเทคโนโลยีการเขียนโปรแกรม แบบแอปพลิเคชันต่างๆ

การที่แอปพลิเคชันต่างๆ จะมีการเรียกใช้หารทำงานของกันและกัน หรือมีการส่งผ่านข้อมูลถึงกันและกัน เป็นสิ่งที่ทำได้ยาก ซึ่งอาจต้องมีการกำหนดอะไรขึ้นมาเองระหว่าง 2 แอปพลิเคชัน นั้นๆ จนกระทั่งในยุคถัดมา ไมโครซอฟท์ได้คิดค้นเทคโนโลยี COM (Component Object Model) เป็นวิธีที่ทำให้เราเขียนโปรแกรมเป็นแบบ Object Oriented และเรียกใช้การทำงานที่มาจากต่างแอปพลิเคชันได้



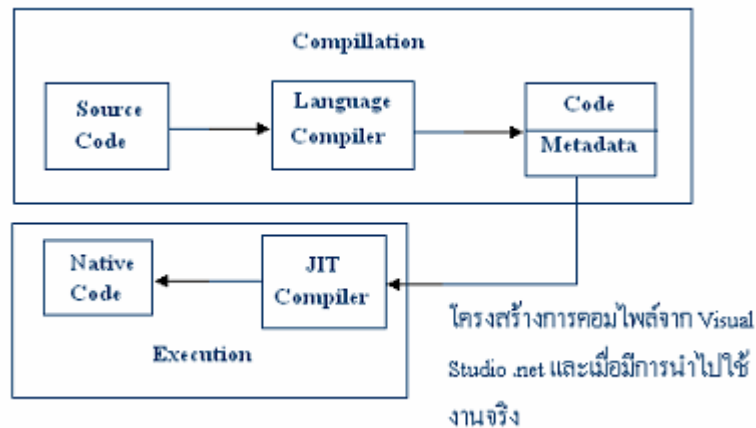
รูปที่ 3-2 พัฒนาการของเทคโนโลยีการเขียนโปรแกรม แบบ *Component Object Model*

หากเราจะอธิบายให้ง่ายขึ้น ก็คือ เปรียบเทียบเหมือนเราเอาแพ็คเกจอันหนึ่งห่อแอปพลิเคชันของเราไว้และการพูดคุยกันของแอปพลิเคชันก็พูดคุยผ่านแพ็คเกจที่เราห่อเอาไว้ จะกระทั่งมาถึงตัว Visual Studio.net ที่ได้รับการออกแบบใหม่ จะเห็นว่าจากรูปเดิม กล่องแพ็คเกจหายไป ถ้าเราพัฒนาด้วยรูปแบบเทคโนโลยี .net นั้น คลาสต่างๆ สามารถติดต่อกันได้โดยตรง



รูปที่ 3-3 พัฒนาการของเทคโนโลยีการเขียนโปรแกรม แบบ *.NET*

การพัฒนาแอปพลิเคชันด้วย Visual Studio .net นั้น เมื่อเราคอมไพล์สิ่งที่เราจะได้ จะไม่ใช่ไค้ดไบนารี (Binary code) เพียงอย่างเดียว แต่จะได้เป็นภาษากลางอันหนึ่งเรียกว่า Microsoft Intermediate Language (MSIL) ซึ่งเป็นภาษาในระดับเลเยอร์ล่างๆ โครงสร้างของภาษา (Syntax) จะเหมือนภาษา Assembly ภายในสิ่งที่เกิดจากการคอมไพล์ก็จะเป็น MSIL ตัวนั้น ภายในตัวมันจะประกอบด้วย 2 ส่วน คือไค้ด กับตัวแอตทริบิวต์หรือ พร็อพเพอร์ตี้ต่างๆ ที่ใช้อธิบายตัวไค้ดนั้นซึ่งเรียกว่า Meta data

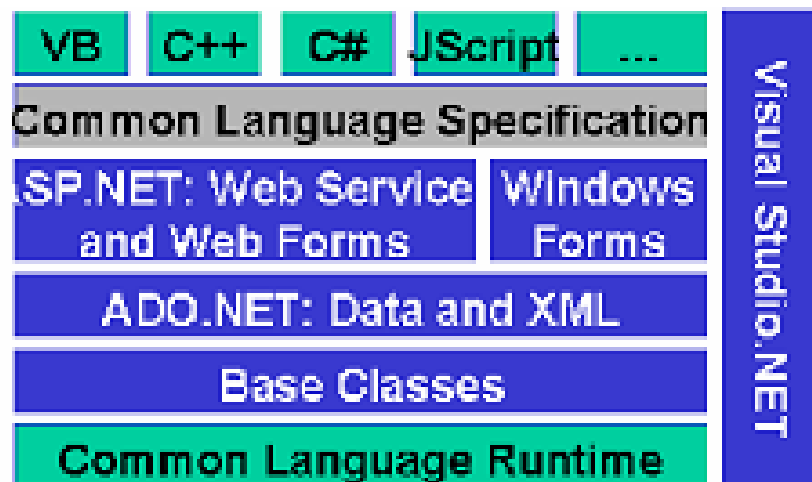


รูปที่ 3-4 โครงสร้างการคอมไพล์จาก Visual Studio.net

จากนั้น เมื่อโค้ดที่เป็น Intermediate Language จะถูกเรียกใช้งานจริงๆ จะมีตัว Compiler (ตัวแปลภาษา) ตัวหนึ่งมาทำการคอมไพล์โค้ดตัวนั้นให้เป็น โค้ด ไบนารี ซึ่ง Compiler ตัวนั้นจะเรียกว่า Just In Time Compiler (JIT Compiler) ที่เรียกว่า Just In Time เพราะว่าจะมีการคอมไพล์เมื่อมีการใช้งาน

ฉะนั้นคลาสหรือโค้ดต่างๆ ที่เราพัฒนาขึ้นแล้วจะถูกคอมไพล์มาเป็น Intermediate Language ที่มีโครงสร้างของภาษาแบบเดียวกัน เพราะฉะนั้นคลาสต่างๆ ในแอปพลิเคชันจึงสามารถทำงานได้อย่าง กลลกกันแลไม่มีข้อขัดข้องอะไร

3.2 สถาปัตยกรรมของ .net



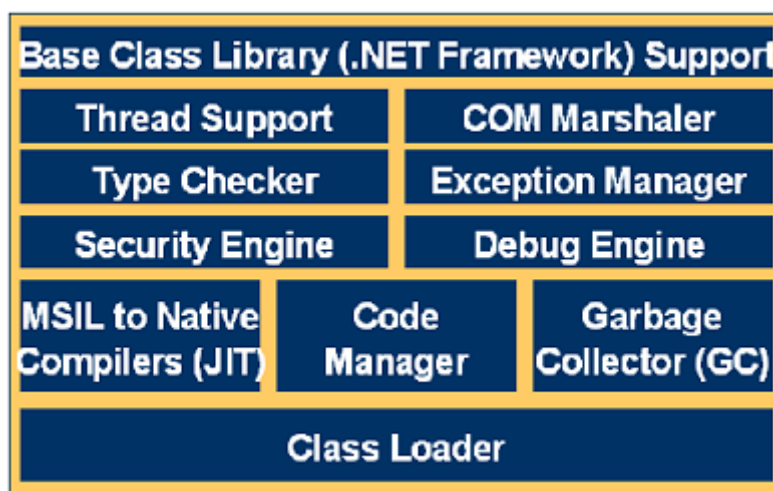
รูปที่ 3-5 โครงสร้างสถาปัตยกรรมของ .net

จากรูปเป็นแสดงถึงสถาปัตยกรรมของแอปพลิเคชัน .net ที่พัฒนาด้วย visual studio.net ถือว่าเป็นการเปลี่ยนแปลงครั้งสำคัญ ทั้งแพลตฟอร์มที่ใช้ในการพัฒนาแอปพลิเคชัน และสถาปัตยกรรมที่ใช้ โดยมีเลเยอร์ล่างสุดคือ .net framework SDK เปรียบเสมือน Runtime Library ที่จะรันอยู่คอย

สนับสนุนการทำงานของแอปพลิเคชัน จากนั้นจะเป็นเลเยอร์ของ Common Language Runtime เป็นผลลัพธ์ของการคอมไพล์ แอปพลิเคชัน .net เลเยอร์ถัดขึ้นมาเป็นเครื่องมือ (Tools) และเทคนิคต่างๆที่เราสามารถใช้พัฒนาแอปพลิเคชันได้ทั้งในเรื่องของ Web Services ,ADO.net ,ASP.net จนกระทั่งถึงเลเยอร์บนสุดคือภาษาที่ใช้ในการพัฒนาแอปพลิเคชันด้วย Visual Studio.net

3.2.1 เลเยอร์ Common Language Runtime

ภายในตัว Common language Runtime จะมีโมดูล (Module) ย่อยๆ ซึ่งเป็นสถาปัตยกรรมภายใน ดังรูป



รูปที่ 3-6 โครงสร้าง เลเยอร์ Common Language Runtime

ด้านล่างสุดจะมี Class Loader ซึ่งเอาไว้โหลดโปรแกรมของเราขึ้นมาทำงาน นอกจากนั้นก็จะมี Compiler ซึ่งจะทำคอมไพล์ภาษา Intermediate Language ให้เป็นภาษาไบনারีโดยจะมี Code manager และ Garbage Collector คอยจัดการกับหน่วยความจำที่เราจองเวลาเรียกใช้ง่าย นอกจากนั้นก็จะมีเรื่องระบบความปลอดภัย (Security) ในการทำงาน รวมทั้ง Debug Engine ในการดัก Runtime Error และตัว Exception Manager การตรวจเช็คชนิดของตัวแปรต่างๆ และด้านบนสุดจะเป็นการใช้งานระหว่าง Library Class ต่างๆ ซึ่งจัดเตรียมมาให้ เพราะฉะนั้นเลเยอร์ของ Common Language Runtime การคอมไพล์แอปพลิเคชันใดก็ตาม ไม่ว่าจะเป็น ASP.net หรือเขียนแอปพลิเคชันบน Windows ธรรมดา หรือจะเป็นการเขียน Web Service ก็ตาม สิ่งที่ได้จากการคอมไพล์จะเป็น Common Language Runtime ตามแผนผังนี้

นั่นคือจากการออกแบบเพื่อสนับสนุนการทำงานร่วมกันใน Common Language Runtime จึงขัดข้อเสียของ COM ไปได้ เนื่องจากของ COM คือเป็นการเอาอะไรบางอย่างมาห่อนๆก็ทำให้เกิดปัญหาเรื่องการเข้ากันได้ (Compatibility) ระหว่างเวอร์ชันเดิมกับเวอร์ชันปัจจุบันแต่เราพัฒนาด้วย Visual Studio .net ข้อเสียของ COM ทั้งหมดก็จะถูกขจัดเสีย

ความจริง Common Language Runtime เป็นหลักการทำงานที่มีวิวัฒนาการมาจาก COM อีกที่หนึ่ง เป็นการ Object Oriented ที่แกนของภาษาเลย โดย Visual Studio .net นั้นถูกออกแบบเพื่อสนับสนุน Object Oriented โดยเฉพาะ คลาสต่างๆที่อยู่ในแต่ละแอปพลิเคชัน A เราอาจจะเขียนด้วยภาษา C++ และแอปพลิเคชัน B ซึ่งเขียนด้วยภาษา VB ได้ คือการ Inheriant ข้ามภาษานั้นสามารถทำได้

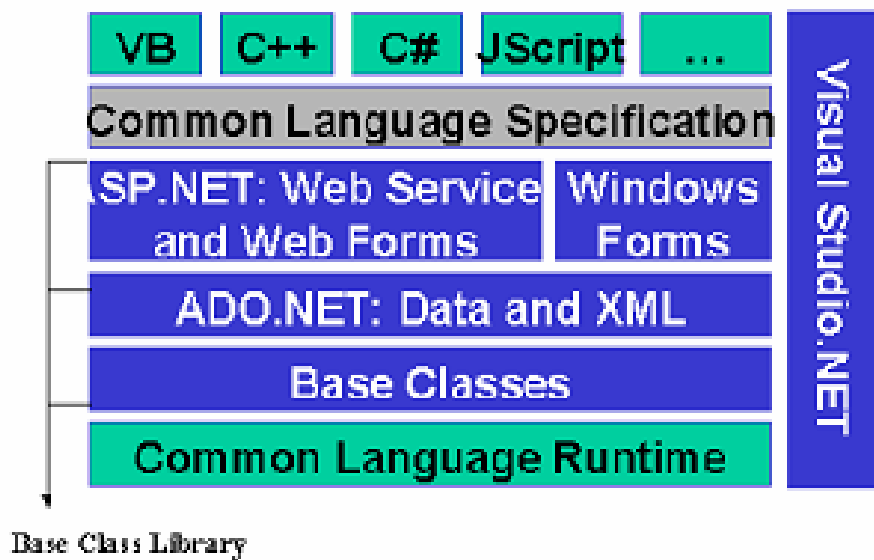


รูปที่ 3-7 การออกแบบเพื่อสนับสนุนการทำงานร่วมกันใน Common Language Runtime

ใน Visual Studio .net จะคอมไพล์เป็นภาษาเดียวกันคือ Intermediate Language ตามรูปก่อนหน้านี้นี้คือคอมไพล์เป็นภาษาอันหนึ่ง เพราะฉะนั้นจึงสามารถ Inherit กันข้ามภาษาได้

นอกจากนี้ ยังสามารถทำงานด้วยกันกับ COM แบบเดิมที่เราเคยเขียนมาแล้วได้ด้วย ซึ่งใช้ Visual Studio เดิม โค้ดเหล่านี้ก็ไม่จำเป็นต้องโยนทิ้งใน Visual Studio .net เราสามารถเรียกใช้งาน COM ได้ตามปกติ และในทางกลับกัน ใน Visual Studio 6.0 ที่มีอยู่ก็สามารถเรียกใช้งานคอมโพเนนต์ที่เขียนด้วย Visual Studio .net ได้เช่นกัน คือเป็นการเข้ากันได้ทั้งสองทาง (Backward-Forward Compatibility) นี่ก็คือข้อดีมากๆ ของ .net ทำให้เราไม่ต้องมาพัฒนาโค้ดใหม่

3.2.2 เลเยอร์ Base Class Library

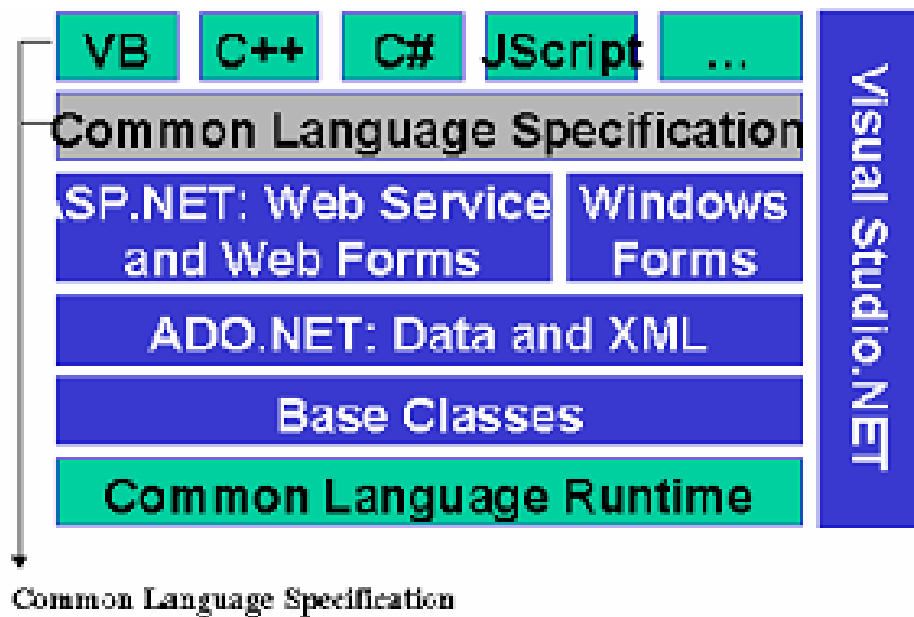


รูปที่ 3-8 โครงสร้าง เลเยอร์ Base Class Library

ตัว Base Class Library ก็คือ การที่เรารวบรวมฟังก์ชัน API (Application programming Interface) ซึ่งกระจัดกระจายอยู่ เวลาจะเรียกใช้เราต้องไปค้นหาใน Help นั่นคือ Base Class Library พยายามที่จะรวบรวม API และฟังก์ชันบางทั้งหมดเกี่ยวกับระบบเข้ามาไว้ในลักษณะของ Object Oriented ทั้งหมด โดยมีคลาสอันหนึ่งเป็นมาตรฐานเป็นคลาสที่สร้างมาในตัวระบบเรียบร้อยแล้ว ซึ่งคลาสทั้งหมดจะอยู่ภายใต้คลาสหลักอันหนึ่งที่เรียกว่า System

ภายในคลาสจะมีคลาสย่อยๆ มากมาย ซึ่งแต่ละอันจะสนับสนุนการทำงานที่เราต้องการได้ ไม่ว่าจะเป็นเรื่องของการทำกราฟฟิค การทำเกี่ยวกับโครงสร้างข้อมูล (Data Structure) การทำเกี่ยวกับเรื่องเครือข่าย (Network) ฟังก์ชัน API เหล่านี้จะถูกจัดกลุ่มให้เป็น Object Oriented อยู่ภายใน System Class การเรียกใช้งาน System Class จะสามารถเรียกได้ VB และ C++

3.2.3 เลเยอร์ Common Language Specification



รูปที่ 3-9 โครงสร้าง เลเยอร์ Common Language Specification

คือเครื่องมือที่ใช้ในการสร้างแอปพลิเคชัน หรือหลักการที่ใช้ในการเขียนโปรแกรมต่างๆ เช่น เรื่องของ ADO.net ,ASP.net ที่ใช้ในการพัฒนาแอปพลิเคชัน แต่สิ่งที่เห็นอย่างทุกอย่างเป็น ภาษาที่เราใช้งาน ภาษาต่างๆ ที่ทำงานใน .net นั้นมีข้อดีคือ ต้องสนับสนุนมาตรฐานเดียวกัน เรียกว่า Common Language Specification

ในไม่ช้าเราจะเห็นเว็บเพจเขียนด้วย COBAL ก็ได้ รวมทั้งภาษาอื่นด้วย นอกจากนี้ใน ตระกูล .net เองเราก็มี VB,VC และ C# และภาษาอื่นๆ เช่น PASCAL,Perl โดยภาษาที่ใช้งานประเภท Object ทั้งหมดสามารถทำเป็น แพลตฟอร์มของ .net ได้ เพราะว่าใน .net นั้นผลิตทุกอย่างเป็น Object Oriented ทั้งหมด